

# Lab: Black Box Testing - Designing & Implementing Test Cases for Loan Eligibility

*This lab provides hands-on experience in applying Black Box Testing techniques to design comprehensive test cases. Students will use Equivalence Partitioning, Boundary Value Analysis, and Decision Table Testing to create tests for a Python loan eligibility calculator and implement these tests using Python's built-in `unittest` module.*

## Lab Overview

Duration: 90 minutes

Type: guided

Language: Python

## Learning Objectives

- • Apply Equivalence Partitioning to identify test classes
- • Use Boundary Value Analysis for edge case testing
- • Design test cases using Decision Table Testing
- • Implement black box test cases using Python assertions
- • Validate both valid and invalid inputs

## Prerequisites

- • Python 3.8+ installed
- • Familiarity with basic Python syntax (functions, conditional statements)
- • Understanding of logical operators (AND, OR, NOT)
- • Basic knowledge of software testing concepts

## Tools Required

### Python (v3.8+)

Category: ToolCategory.CLI\_TOOL

Installation: Follow instructions at <https://www.python.org/downloads/>

Verification: `python --version`

Documentation: <https://docs.python.org/3/>

## Text Editor / IDE (vAny modern IDE (e.g., VS Code, PyCharm))

Category: ToolCategory.IDE

Installation: Not required to install a specific one, use your preferred editor.

Verification: N/A

Documentation: <https://code.visualstudio.com/>

## Setup Instructions

### Step 1: Create Lab Directory

Create a new directory for your lab work. This will keep your files organized.

Commands:

```
mkdir black_box_testing_lab  
cd black_box_testing_lab
```

*Expected Output: A new directory named 'black\_box\_testing\_lab' is created and you are inside it.*

⚠ Troubleshooting: If 'mkdir' fails, check your current directory permissions. If 'cd' fails, ensure the directory was created correctly.

### Step 2: Create the System Under Test (SUT) File

Create the Python file containing the `is\_eligible\_for\_loan` function that you will be testing. This function simulates a loan eligibility calculator.

Commands:

```
touch loan_calculator.py  
Open loan_calculator.py in your text editor and paste the provided starter code.
```

*Expected Output: An empty file named 'loan\_calculator.py' is created.*

⚠ Troubleshooting: If 'touch' fails, ensure you are in the correct directory. If you cannot save the file, check permissions.

### Step 3: Create the Test File Structure

Create the Python file where you will write your test cases. We will use Python's built-in `unittest` module.

Commands:

```
touch test_loan_eligibility.py  
Open test_loan_eligibility.py in your text editor and paste the provided starter code.
```

*Expected Output: An empty file named 'test\_loan\_eligibility.py' is created.*

⚠ Troubleshooting: Same as Step 2.

## Lab Tasks

### Task 1: Understand the System Under Test (SUT)

🕒 Time: 10 min | 🎯 Difficulty: easy

Familiarize yourself with the `is_eligible_for_loan` function and its business rules.

Understanding the requirements is crucial for effective test design.

#### Instructions:

1. Read through the `loan_calculator.py` code carefully.
2. Identify all input parameters and their constraints (e.g., age range, income minimum, credit score range).
3. Note down the conditions for loan eligibility and how employment status and customer type affect the discount.

💡 Hints: • Pay close attention to the `if` and `elif` conditions within the function.

✅ Expected Outcome: A clear understanding of the loan eligibility rules and the function's behavior.

### Task 2: Apply Equivalence Partitioning (EP)

🕒 Time: 15 min | 🎯 Difficulty: medium

Apply Equivalence Partitioning to the numerical input parameters (Age, Income, Credit Score) to identify valid and invalid equivalence classes.

#### Instructions:

1. For each numerical input parameter (age, income, credit\_score), identify at least one valid and one invalid equivalence class.
2. Document your findings in a table format (e.g., in a `analysis.md` file).
3. Example table structure:
4. ```
5. Parameter | Valid Equivalence Classes | Invalid Equivalence Classes
6. -----|-----|-----
7. Age | [18-65] | <18, >65
8. Income | [>=30000] | <30000
9. Credit | [300-850] | <300, >850
10. ```

11. Select one representative value from each identified equivalence class to form initial test cases.

💡 Hints: • Think about what values should produce a 'true' outcome and what values should produce a 'false' outcome for each parameter's constraint.

- Consider edge cases for invalid classes, e.g., what if age is exactly 0 or 100?

✅ Expected Outcome: A documented table of equivalence classes for Age, Income, and Credit Score, with representative test values.

### Task 3: Apply Boundary Value Analysis (BVA)

🕒 Time: 15 min | 🎯 Difficulty: medium

Apply Boundary Value Analysis to the numerical input parameters (Age, Income, Credit Score) to identify critical boundary values for testing.

#### Instructions:

1. For each numerical input parameter (age, income, credit\_score), identify boundary values including: minimum, minimum-1, maximum, maximum+1 (where applicable).
2. Document your findings in a table format (e.g., in `analysis.md`).
3. Example table structure:
4. ```
5. Parameter | Boundary Values (Valid) | Boundary Values (Invalid)
6. -----|-----|-----
7. Age | 18, 65 | 17, 66
8. Income | 30000 | 29999
9. Credit | 300, 850 | 299, 851
10. ```
11. These boundary values will form additional, crucial test cases.

💡 Hints: • BVA focuses on values \*at\* and \*just outside\* the boundaries of valid ranges.

- Ensure you consider both lower and upper bounds for each range.

✅ Expected Outcome: A documented table of boundary values for Age, Income, and Credit Score.

### Task 4: Apply Decision Table Testing (DTT)

🕒 Time: 20 min | 🎯 Difficulty: hard

Apply Decision Table Testing to the complex business rules involving `employment\_status` and `customer\_type` to ensure all combinations are covered.

### Instructions:

1. Focus on the part of the `is\_eligible\_for\_loan` function that determines the final eligibility and discount based on `employment\_status` and `customer\_type`.
2. Identify all conditions (inputs) and actions (outcomes) related to these rules.
3. Construct a decision table. Consider conditions like 'employed', 'unemployed', 'new customer', 'existing customer', 'premium customer'.
4. Determine the corresponding actions (e.g., 'eligible', 'ineligible', 'discount applied', 'no discount').
5. Document your decision table (e.g., in `analysis.md`).
6. Example structure (simplified):
7. ```
8. Conditions / Actions | Rule 1 | Rule 2 | Rule 3 | ...
9. -----|-----|-----|-----|-----
10. Employed | Y | Y | N | ...
11. Premium Customer | Y | N | - | ...
12. -----|-----|-----|-----|-----
13. Eligible | X | X | | ...
14. Discount | X | | | ...
15. ```
16. Each rule in your decision table will represent a unique test case.

- 💡 Hints:
- Break down the complex logic into individual conditions and their possible states.
  - A 'don't care' (-) value can simplify rules where a condition's state doesn't affect the outcome.
  - Ensure your table covers all possible combinations, even if some lead to 'ineligible'.

✅ Expected Outcome: A comprehensive decision table covering combinations of employment status and customer type, leading to specific eligibility and discount outcomes.

## Task 5: Implement Test Cases using Python `unittest`

🕒 Time: 20 min | 🎯 Difficulty: medium

Translate the test cases identified in EP, BVA, and DTT into executable Python tests using the `unittest` framework.

### Instructions:

1. Open `test\_loan\_eligibility.py`.
2. Import the `is\_eligible\_for\_loan` function from `loan\_calculator.py`.

3. Inside the `TestLoanEligibility` class, create individual test methods (e.g., `test_age_valid_middle`, `test_age_below_min`, `test_credit_score_boundary_max_plus_one`).
4. For each test method, call `is_eligible_for_loan` with your chosen parameters and use `self.assertTrue()` or `self.assertFalse()` to check the expected boolean outcome.
5. Aim for 15-20 test cases, ensuring coverage from your EP, BVA, and DTT analyses.
6. Example test case:
7. 

```
python
```
8. 

```
def test_age_valid_middle(self):
```
9. 

```
    # EP: Valid age (middle of range)
```
10. 

```
    self.assertTrue(is_eligible_for_loan(age=40, income=50000, credit_score=700,
```
11. 

```
                    employment_status='employed', customer_type='new'))
```

💡 Hints:

- Name your test methods descriptively to indicate what scenario they are testing.
- Use comments to link tests back to specific EP, BVA, or DTT rules.
- Remember to consider all parameters for each test, even if focusing on one for a specific technique.

✅ Expected Outcome: A `test_loan_eligibility.py` file containing 15-20 well-structured test methods that assert the correct behavior of `is_eligible_for_loan`.

## Task 6: Run and Validate Tests

🕒 Time: 10 min | 🌟 Difficulty: easy

Execute your test suite and interpret the results. Ensure all your implemented tests pass.

### Instructions:

1. From your terminal in the `black_box_testing_lab` directory, run your tests using the command: `python -m unittest test_loan_eligibility.py`.
2. Review the output. All tests should pass. If any fail, carefully examine the test case and the `is_eligible_for_loan` function logic to identify the discrepancy.
3. Modify either your test case or the SUT until all tests pass and reflect the correct business logic.

💡 Hints:

- If a test fails, first check if your expected outcome for that specific input was correct. Then, review the function's logic.
- Use print statements inside your `is_eligible_for_loan` function temporarily to debug values if needed.

✓ Expected Outcome: All 15-20 implemented test cases pass successfully, indicating correct test design and function implementation.

## Checkpoints

### Checkpoint 1: EP Analysis Complete

Success Criteria: Equivalence Partitioning table is clearly documented with appropriate classes and values.

Verification Steps:

- Review the `analysis.md` file (or equivalent).
- Check that Age, Income, and Credit Score each have at least one valid and one invalid equivalence class identified.
- Ensure representative values are chosen for each class.

### Checkpoint 2: BVA Analysis Complete

Success Criteria: Boundary Value Analysis table is clearly documented with correct boundary values.

Verification Steps:

- Review the `analysis.md` file.
- Check that Age, Income, and Credit Score each have minimum, minimum-1, maximum, and maximum+1 values identified (where applicable).

### Checkpoint 3: DTT Analysis Complete

Success Criteria: Decision Table is comprehensive and accurately reflects the business rules.

Verification Steps:

- Review the `analysis.md` file.
- Check that a decision table is created for `employment\_status` and `customer\_type`.
- Ensure all relevant conditions and actions are covered, and rules are distinct.

### Checkpoint 4: All Tests Pass

Success Criteria: The terminal output indicates that all implemented tests passed successfully (e.g., 'Ran X tests in Ys OK').

Verification Steps:

- Run `python -m unittest test\_loan\_eligibility.py` in the terminal.
- Observe the output.

## Starter Code

 **loan\_calculator.py** The Python function to be tested, implementing loan eligibility rules.

```
'''
This module contains the loan eligibility calculator function.
'''

def is_eligible_for_loan(age: int, income: int, credit_score: int, employment_status:
str, customer_type: str) -> bool:
    """
    Determines if a person is eligible for a loan based on several criteria.

    Args:
        age (int): The applicant's age.
        income (int): The applicant's annual income.
        credit_score (int): The applicant's credit score.
        employment_status (str): The applicant's employment status (e.g., "employed",
"unemployed", "student").
        customer_type (str): The applicant's customer type (e.g., "new", "existing",
"premium").

    Returns:
        bool: True if the applicant is eligible for a loan, False otherwise.
    """

    # Rule 1: Age must be between 18-65
    if not (18 <= age <= 65):
        return False

    # Rule 2: Income must be >= 30000
    if income < 30000:
        return False

    # Rule 3: Credit score between 300-850
    if not (300 <= credit_score <= 850):
        return False

    # Rule 4: Employment status affects eligibility
    if employment_status.lower() not in ["employed", "self-employed"]:
        # Unemployed or student might be eligible with other strong factors, but for
this simple model, let's say no.
        return False

    # If all basic conditions are met, determine final eligibility and apply discount
logic
    # Discount based on customer type (new, existing, premium)
    # For this lab, we
```

 **test\_loan\_eligibility.py** The Python test file template using `unittest` where students will write their test cases.

```
'''
```

This module contains the unit tests for the loan\_calculator.py module.  
Students will add their black box test cases here.

```
'''

import unittest
from loan_calculator import is_eligible_for_loan

class TestLoanEligibility(unittest.TestCase):
    """
    Test suite for the is_eligible_for_loan function using Black Box Testing
    techniques.
    """

    # --- Example Test Cases (Students will add many more based on EP, BVA, DTT) ---

    def test_valid_applicant_employed_new(self):
        """
        Test case for a valid applicant who is employed and a new customer.
        Expected: True (eligible)
        """
        self.assertTrue(is_eligible_for_loan(age=30, income=45000, credit_score=750,
        employment_status='employed', customer_type='new'))

    def test_invalid_age_below_min(self):
        """
        Test case for an applicant with age below the minimum required (EP/BVA).
        Expected: False (ineligible)
        """
        self.assertFalse(is_eligible_for_loan(age=17, income=45000, credit_score=750,
        employment_status='employed', customer_type='new'))

    # TODO: Add many more test cases here based on your EP, BVA, and DTT analysis!

if __name__ == '__main__':
    unittest.main()
```

 **analysis.md** Template for documenting Equivalence Partitioning, Boundary Value Analysis, and Decision Table Testing results.

# Black Box Testing Analysis for Loan Eligibility Calculator

This document outlines the Equivalence Partitioning, Boundary Value Analysis, and Decision Table Testing applied to the `is\_eligible\_for\_loan` function.

## ## 1. Equivalence Partitioning (EP)

Identify valid and invalid equivalence classes for each input parameter.

### Age

| Equivalence Class | Description                    | Representative Value |
|-------------------|--------------------------------|----------------------|
| Valid (18-65)     | Ages within the eligible range | `40`                 |
| Invalid (<18)     | Ages below the minimum         | `10`                 |

|               |                        |      |  |
|---------------|------------------------|------|--|
| Invalid (>65) | Ages above the maximum | `70` |  |
|---------------|------------------------|------|--|

### ### Income

| Equivalence Class | Description                 | Representative Value |  |
|-------------------|-----------------------------|----------------------|--|
| -----             | -----                       | -----                |  |
| Valid (>=30000)   | Incomes at or above minimum | `50000`              |  |
| Invalid (<30000)  | Incomes below minimum       | `20000`              |  |

### ### Credit Score

| Equivalence Class | Description                  | Representative Value |  |
|-------------------|------------------------------|----------------------|--|
| -----             | -----                        | -----                |  |
| Valid (300-850)   | Scores within eligible range | `600`                |  |
| Invalid (<300)    | Scores below minimum         | `250`                |  |
| Invalid (>850)    | Scores above maximum         | `900`                |  |

## ## 2. Boundary Value Analysis (BVA)

Identify boundary values for each numerical input parameter.

### ### Age

| Boundary Value | Description | Expected Outcome |  |
|----------------|-------------|------------------|--|
| -----          | -----       | -----            |  |
| `17`           | Min - 1     | Ineligible       |  |
| `18`           | Min         | Eligible         |  |
| `65`           | Max         | Eligible         |  |
| `66`           | Max + 1     | Ineligible       |  |

### ### Income

| Boundary Value | Description | Expected Outcome |  |
|----------------|-------------|------------------|--|
| -----          | -----       | -----            |  |
| `29999`        | Min - 1     | Ineligible       |  |
| `30000`        | Min         | Eligible         |  |

### ### Credit Score

| Boundary Value | Description | Expected Outcome |  |
|----------------|-------------|------------------|--|
| -----          | -----       | -----            |  |
| `299`          | Min - 1     | Ineligible       |  |
| `300`          | Min         | Eligible         |  |
| `850`          | Max         | Eligible         |  |
| `851`          | Max + 1     | Ineligible       |  |

## ## 3. Decision Table Testing (DTT)

Analyze rules involving `employment\_status` and `customer\_type`.

#### \*\*Conditions:\*\*

- C1: Age, Income, Credit Score are all valid (pre-conditions met)
- C2: Employment Status is 'employed' or 'self-employed'
- C3: Customer Type is 'premium'
- C4: Customer Type is 'existing' (and not premium)

#### \*\*Actions:\*\*

- A1: Eligible for Loan
- A2: Apply Premium Discount (e.g., 5% off interest)
- A3: Apply Existing Customer Discount (e.g., 2% off interest)

| Rule # | C1 | C2 | C3 | C4 | A1 | A2 | A3 | Notes                                     |
|--------|----|----|----|----|----|----|----|-------------------------------------------|
| 1      | Y  | Y  | Y  | N  | X  | X  |    | Employed, Premium Customer                |
| 2      | Y  | Y  | N  | Y  | X  |    | X  | Employed, Existing Customer               |
| 3      | Y  | Y  | N  | N  | X  |    |    | Employed, New Customer                    |
| 4      | Y  | N  | -  | -  |    |    |    | Not Employed (e.g., unemployed, student)  |
| 5      | N  | -  | -  | -  |    |    |    | Age/Income/Credit invalid (any one fails) |

\*Note: For the purpose of this lab, the `is_eligible_for_loan` function only returns `True` or `False`. The

## Common Issues & Troubleshooting

**\*\*Incorrect Equivalence Classes/Boundaries:\*\*** Students might misidentify valid/invalid ranges or boundary values. Advise them to re-read the function's constraints carefully.

**\*\*Incomplete Decision Tables:\*\*** Missing combinations of conditions, leading to uncovered scenarios. Encourage systematic enumeration of all condition states.

**\*\*Incorrect Assertions:\*\*** Using `assertTrue` when `assertFalse` is expected, or vice-versa. Stress the importance of knowing the *expected* outcome for each test case.

**\*\*Not Importing SUT:\*\*** Forgetting to `from loan_calculator import is_eligible_for_loan` in the test file.

**\*\*Test Methods Not Starting with `test_`:** `unittest` only discovers methods prefixed with `test_`.

## Grading Rubric

This lab will be graded based on the following criteria:

- **\*\*Completeness of Analysis (40%):\*\*** Accuracy and thoroughness of Equivalence Partitioning, Boundary Value Analysis, and Decision Table Testing documentation.
- **\*\*Test Case Coverage (40%):\*\*** Number and quality of implemented test cases in `test_loan_eligibility.py`, ensuring they effectively cover all identified equivalence classes, boundary values, and decision table rules.
- **\*\*Correctness of Implementation (20%):\*\*** All implemented test cases pass successfully, demonstrating correct assertions and understanding of expected outcomes.

## Submission Requirements

- • loan\_calculator.py (final version after any necessary bug fixes)
- • test\_loan\_eligibility.py (containing all your implemented test cases)
- • analysis.md (or similar document) containing your EP, BVA, and DTT tables/analysis.